ARTICLE | Article in Press

Hierarchical graph attention and memory-augmented learning for spatio-temporal network intrusion detection

Srinivasa Rao Nune · Devi Uma Moka · G. Gifta Jerith · Pattlola Srinivas · Sreelakshmi Udata

Received: 15 June 2026 / Accepted: 26 October 2026
© The Author(s) 2026

Abstract

Graph neural networks are increasingly used for network intrusion detection, but their reported advantages often depend on evaluation choices. We present STAR-IDS, which builds a graph from every window of ten consecutive network flows, applies two-level graph attention and a prototype memory bank, and estimates prediction uncertainty with Monte Carlo Dropout. STAR-IDS was evaluated on NF-CSE-CIC-IDS2018-v3, with real IP-address nodes and flows ordered by start time, and on the IoT dataset NF-ToN-IoT-v2, using block-wise splits that prevent leakage, temporally blocked cross-validation, five paired seeds and controlled baselines (E-GraphSAGE, GAT, GCN, GraphSAGE and a graph-free MLP) trained under identical conditions. On NF-CSE-CIC-IDS2018, STAR-IDS achieved 87.61%±1.22% accuracy and 86.41% macro-F1, and no difference from the baselines or ablations was statistically significant after Holm correction; on NF-ToN-IoT, the graph-free MLP was the most accurate model (81.10% vs. 77.21%). The uncertainty estimates separated erroneous from correct predictions with an AUC of 0.89, and on that dataset flagged predictions were 16–25 percentage points less accurate than auto-accepted ones. Control experiments show that shuffling flows before windowing, synthetic node identifiers and forward-in-time splits change accuracy by up to 68 percentage points.

Keywords Intrusion detection system · GNN · Attention · Memory augmented learning · XAI

Introduction

The rapid rise in the number of devices connected to the network and the growing complexity of cyber-attacks have made network security one of the most pressing issues in contemporary computing infrastructure. Intrusion Detection Systems (IDS) are the first line of defense against malicious activities, constantly scanning network traffic to detect and prevent potential security breaches. Conventional signature-based IDS techniques, although successful in detecting known attack patterns, have difficulty detecting unknown or zero-day attacks that do not conform to predefined signatures. This has led to the research community exploring the development of intelligent and adaptive detection systems that can learn and detect new attack patterns in real-time.

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.



https://doi.org/10.1038/s41598-026-78200-2

Published online: 07 November 2026

Machine learning algorithms like Random Forests, Support Vector Machines (SVMs), and Decision Trees have been extensively employed for intrusion detection tasks[1–6], and moderate success has been obtained in pattern recognition and anomaly detection tasks. However, traditional machine learning algorithms are known to be vulnerable to heavy manual feature engineering and are not scalable for high-dimensional data and complex patterns of attacks. Recent advancements in deep learning have completely changed the scenario for network intrusion detection systems, and deep learning algorithms like Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Long Short-Term Memory (LSTM) networks have shown promising results by learning hierarchical feature representations of raw network traffic data automatically. However, these traditional deep learning models have largely been concerned with modeling the network traffic as sequential or tabular data, without taking into consideration the natural graph structure of network communications. Network traffic has a natural graph structure, where the nodes are the network entities (hosts, servers, or devices), and the edges are the communication flows between them. This is particularly true in the case of complex networks of cyber-physical systems, where network attacks are spatial–temporal in nature: the attackers are expected to exploit the structural weaknesses of the network topology as well as the temporal correlation of the communication flows, thus requiring a modeling paradigm that can simultaneously capture both of these properties.

Graph Neural Networks (GNNs) models have been providing promising techniques to learn on graph-structured data, allowing to model node features and edge relationships through message passing. Although contemporary approaches in intrusion detection have focused on GNN-based approaches[7,8] (GNN, GCN, GAT, etc.), the attention improvement of GNN approaches has recently been subjected to a number of studies. On the other hand, current GNN-based IDS methods also suffer from a number of key drawbacks. First, all the existing methods take static graph construction approaches that lack the ability to indicate the dynamic and changing nature of network communications. Second, the existing methods all utilize one layer of attention, which is unable to model hierarchical and multi-scale patterns often contained in complex attack behaviors. In addition to this, none of the existing approaches provide good mechanisms to retain and reuse historic attack patterns thus limiting their detection of attack behavior that may evolve over time. Lastly, the black-box nature of deep learning models greatly raises concerns about the trustworthiness and explainability of deep learning techniques, especially in the field of security applications where knowing how the results of detection are reasoned is extremely important.

To address these issues, we propose STAR-IDS (Spatio-Temporal Attack Recognition for Intrusion Detection System), which combines dynamic graph construction, hierarchical graph attention and memory-based learning with uncertainty estimation for analyst triage. This work makes four contributions, listed below; Section "Results and analysis" assesses how far the evidence supports each of them. First, network traffic is represented as a sequence of graphs, each describing the communication among hosts within a short window of consecutive flows, so that flows occurring together are analyzed jointly; whether this context improves detection is tested in Section "Results and analysis".

Contributions

- A leakage-controlled graph construction and evaluation pipeline with real endpoint (IP) nodes, chronological windows, block-wise splitting, training-only preprocessing and rare-class-priority window labeling.
- A controlled comparison of STAR-IDS, its ablations, E-GraphSAGE, GAT, GCN, GraphSAGE and a graph-free MLP on NF-CSE-CIC-IDS2018 and the IoT dataset NF-ToN-IoT with five paired seeds and temporally blocked cross-validation, showing that neither the hierarchical attention nor the memory bank yields a statistically significant improvement.
- A quantitative analysis of evaluation practices common in existing graph-based IDS work (shuffled windows, synthetic node identifiers and forward-in-time splits), which change the reported accuracy by up to 68 percentage points.

- A Monte Carlo Dropout uncertainty mechanism whose estimates separate erroneous from correct predictions (AUC 0.81–0.89) and support routing uncertain predictions to analysts.

Related work

The growing number of cyber threats has led to the need for the development of advanced intrusion detection systems (IDS) using machine learning (ML) and deep learning (DL) techniques. Recent studies show a wide range of approaches being used to tackle the issues of network security in a novel manner. The literature reviewed below spans five broad categories: feature-space and dimensionality-reduction approaches, deep and hybrid neural architectures, graph-based topological modeling, distributed and privacy-preserving learning, and class-imbalance handling; STAR-IDS is positioned at the intersection of the graph-based and distributed-imbalance-aware categories, combining elements neither category addresses alone. Several recent surveys summarize this body of work: Walling and Lodh[1] review machine learning and deep learning techniques for network intrusion detection in the IoT, Al-Haija and Droos[2] survey deep learning-based IDS for IoT, Noor et al.[3] review machine learning and transfer learning strategies for IDS in 5G and beyond networks, Fares et al.[4] compare machine learning, deep learning and ensemble approaches, Dong and Kotenko[5] analyze the impact of machine learning on intrusion detection in the AI era, and Hozouri et al.[6] survey IDS advances together with emerging cybersecurity challenges. For flow-based evaluation, standardized NetFlow feature sets[9] and the temporal properties of NetFlow datasets[10] are particularly relevant to this work, whereas Mohammed and Fourati[11] examine datasets for intrusion detection in intra- and inter-UAV communication systems.

Dimensionality reduction in feature space is an essential preprocessing task in IDS design. Madhloom Kurdi et al.[12] used the Cuckoo Search algorithm to reduce feature dimensionality from 49 to 22 for the KDD and 49 to 19 for the UNSW-NB15 datasets. Their design of the Random Forest, Quadratic Discriminant Analysis, and Linear Discriminant Analysis classifiers reported accuracy values above 0.90 for both datasets, proving the success of metaheuristic optimization in feature selection. Likewise, Ahmed et al.[13] used fuzzy clustering for dimensionality reduction prior to training a two-layer LSTM network with 20 units in each layer on the UNSW dataset, reporting an accuracy of 0.96. Data quality is a related preprocessing concern: Tahir et al.[14] proposed an approach for handling missing data to enhance network intrusion detection.

Some specialized and deep learning models have shown a great success for detecting complex patterns on network traffic datasets. Kamalakkannan et al.[15] applied a Bi-LSTM to normalized KDD data without null values and without class balancing and reported an accuracy of 0.81. Berguiga et al.[16,17] proposed a hybrid CNN-LSTM model evaluated on the IoTID20 and Edge-IIoTset datasets with an accuracy of 0.99, and Susilo et al.[18] proposed a hybrid deep learning IDS against various attacks. These hybrid models merge the pattern recognition ability of LSTMs and the spatial feature extraction strength of CNNs. Further deep and hybrid architectures include the hybrid CNN-LSTM intrusion detection approach of Bamber et al.[19], the optimized LSTM-based anomaly detection model of Dash et al.[20], the HAE-HRL system of Xue et al.[21], which combines a novel autoencoder with a hybrid enhanced LSTM-CNN residual network, the deep learning IDS for IoT networks of Selem et al.[22], and the efficient deep neural network evaluated on CIC-IDS-2017 by Bandarupalli[23]. Chen et al.[24] addressed dynamic IoT environments with synaptic intelligent convolutional neural networks, Serrano[25] integrated artificial intelligence into an IoT intrusion detection system (CyberAIBot), and Sharma et al.[26] enhanced IDS with adaptive neuro-fuzzy inference systems. Taken together, these hybrid models[19,21,27] treat flows or records as independent samples, which is the assumption that graph-based approaches relax.

Building on this paradigm shift, Kamal and Mashaly[27] proposed hybrid DL models, which incorporate autoencoders and transformers wherein feature extraction was performed at the middle layer using DNN, for binary and multi-class classification tasks on the CICIDS2017 and NF-BoT-IoT-v2 datasets, achieving an accuracy of

0.99 and 0.97 respectively. Transformers are a crucial progression for the establishment of long-range dependencies in network traffic modelling;

Graph neural network applied to IDS, Deng and Huang[7] adapted the styles of multi-hop attention mechanism to model topological relationships in the network data. By modeling the relational structure of flow communications, this work represented a paradigm shift away from the independent samples assumption that traditional models of network traffic required and achieved a significant improvement in accuracy. Graph-based IDS of this kind[7,8] are the closest related work to STAR-IDS.

Other hot spots of IDS research include Privacy-preserving learning, as well as Distributed learning. Olanrewaju-George and Pranggono[28] used federated learning with DNN classifiers and reached 0.90 accuracy on 7,062,606 samples of Mirai and BASHLITE datasets, using 115 features. Sudha and Bolla[29] proposed a federated intrusion forecasting framework (Puffer Fish) that reached 0.99 accuracy on NSL-KDD. Such federated methods enable joint model training on distributed networks without data privacy leakage, which is a key concern in enterprise and IoT networks. Federated approaches[28,29] are complementary to the centralized graph-based detection studied here.

Binary Classification on Quantum Machine Learning for IDS Kumar and Swarnkar[30] introduced quantum support vector machines for IDS and demonstrated that quantum machine learning achieved an accuracy of 0.91 with significantly less data than classical ML and DL models. This can be very promising for resource-scarce scenarios where obtaining data, or computational resources is an issue.

Optimization techniques have been widely used to improve the performance of models. Benmalek and Seddiki[31] used particle swarm optimization with SVM, KNN, CatBoost, and LSTM models on RT_IoT2022 datasets, obtaining 0.96 accuracy with LSTM. Manivannan and Senthilkumar[32] designed an adaptive RNN with fox optimization and co-occurrence matrix feature extraction, experimenting with different feature sets for better results. Sheelavant et al.[33] used the Cuckoo Search optimizer in an ensemble technique on Kyoto network datasets with 91,000 samples, obtaining 0.98 accuracy. Shi et al.[34] used an artificial hummingbird algorithm with binary variant approach, obtaining 0.99 accuracy on NSL-KDD, CIC-IDS2017, and CSE-CIC-IDS2018 datasets. Metaheuristic optimization therefore remains a common way to tune IDS models[31,32]. Ensemble learning has also been applied in real-time settings: Alalwany et al.[35] proposed stacking ensemble deep learning for real-time intrusion detection in Internet of Medical Things environments.

Beyond network-traffic-based intrusion detection specifically, related security domains offer complementary techniques and highlight STAR-IDS's positioning. Yang et al.[36] proposed FuzzCoder, a code large language model fine-tuned to predict effective byte-level mutation positions and strategies for fuzz testing, framing input mutation as a sequence-to-sequence learning problem rather than the random mutation used by traditional fuzzers such as AFL. This differs fundamentally from STAR-IDS in both target (pre-deployment vulnerability discovery in program binaries, versus post-deployment detection of malicious traffic) and technique (LLM-guided sequence generation, versus graph-structured attention over network flows), though both share the broader goal of using learned models to replace hand-crafted heuristics in security tooling. Yang et al.[37] proposed a Conditional Deep Belief Network-based mechanism for real-time wireless intrusion detection, addressing class imbalance via a window-based instance selection algorithm (SamSelect) and dimensionality reduction via a Stacked Contractive Auto-Encoder; this is the most directly comparable of the three works, since it also targets real-time network intrusion detection under class imbalance, but it treats traffic instances independently rather than modeling the graph-structured relationships between communicating hosts that are central to STAR-IDS's design, and does not incorporate uncertainty quantification for analyst triage. Together, these two works illustrate that learned, data-driven approaches are displacing heuristic methods across the security tooling landscape broadly, while underscoring that graph-structured, uncertainty-aware modeling of network traffic specifically remains comparatively underexplored, which is the gap STAR-IDS addresses.

Data imbalance can be handled using synthetic data generation, which has shown positive results. Das et al.[38] employed the generation of synthetic data through conditional GANs to train hybrid FNN-CNN models. Rahman et al.[39] proposed the integration of SVM and GAN models on UNSW-NB15 datasets, reporting 0.98

accuracy, thus proving that generative models can improve classification accuracy by providing balanced training datasets. The growing need for interpretability in models has resulted in the integration of explainable AI in IDS. Mahmoud et al.[40] proposed a two-tier IDS using SHAP-based explainable AI, reporting 0.99 accuracy on UNSW-NB15 and CICIDS2017 datasets. Kharoubi et al.[41] and Wali et al.[42] both proposed the use of explainable AI-based ML models on CSE-CIC-IDS2018 datasets, analyzing results in terms of interpretability. This is in line with the vital need for transparent decision-making in security-critical systems, where the rationale behind attack detection is crucial for responding to incidents and improving systems. Beyond post-hoc explanation, Cerasuolo et al.[43] proposed adaptable, incremental and explainable network intrusion detection systems for the Internet of Things.

Methodology

Dynamic graph construction

The STAR-IDS model is designed to produce the mapping of raw data relating to network traffic directly extracted from flow-level exports onto graphs that capture spatial and temporal patterns of network communication, rather than treating each flow as an independent classification instance. Figure 1 summarizes the architecture and the processing pipeline.

For each temporal window t, we build a directed multigraph $G_t = (V_t, E_t, X_t, A_t)$, where V_t is the set of nodes, E_t the set of edges (one per flow), X_t the node features and A_t the edge attributes. A window consists of $W = 10$ consecutive flows in chronological order (flow start time where available, otherwise the original file order; rows are never shuffled), with a stride of 10 flows (zero overlap). The nodes are the distinct source and destination IP addresses occurring in the window (between 2 and 20 nodes), so each edge connects the two communicating hosts of a flow and the graph represents the host-level communication topology of that time slice. Multiple flows between the same pair of hosts are kept as parallel edges. When a dataset release contains no IP addresses, a synthetic node mapping is used instead: $n_{src} = \lfloor f_1 \times 1000 \rfloor \bmod 150$ and $n_{dst} = \lfloor f_2 \times 1000 \rfloor \bmod 150 + 150$, where f_1 and f_2 are the first two raw (unscaled) feature columns. This mapping is synthetic: it does not identify hosts, and Section "Effect of the evaluation protocol and of the graph semantics" quantifies its collisions and its effect.

For each edge e_ij connecting source node i to destination node j, we associate a feature vector a_e of NetFlow flow statistics (47 features for NF-CSE-CIC-IDS2018 and 39 for NF-ToN-IoT; Table 1). IP addresses, ports, timestamps and labels are excluded from the features. The initial feature of a node is the mean of its outgoing plus the mean of its incoming edge features (Eq. (8)).

The label of the graph for each temporal window is decided by a rare-class-priority rule rather than plain majority voting; the class priority order is computed from the class frequencies of the training partition only. A naive majority-vote rule requires a class to constitute more than half of a window's flows to become that window's label; for a class present at, for example, 3% of overall flows, the probability that 5 or more of 10 randomly-arranged flows in a window belong to that class is on the order of 10^-6 by direct binomial calculation, meaning such a class would almost never become a window's label under plain majority voting regardless of how much data exists for it. Instead, a window is labeled with the rarest class present in it at all (at least one flow), falling back to the majority class only when no rarer class is present; this reflects standard practice in security-focused window labeling, where a window containing any attack-class flows should be flagged as that attack rather than diluted toward the majority (typically benign) label, since under-flagging a rare attack is a more costly error than a conservatively-labeled window.

Graph construction is also designed to prevent leakage. Windows do not overlap, and the flow sequence is divided into contiguous blocks of 500 flows (50 windows) before any window is built; whole blocks are assigned to the training, validation or test partition, so no flow, window or block appears in more than one partition. To keep every class represented while preserving temporal locality, blocks are allocated class-wise, starting with the

https://doi.org/10.1038/s41598-026-78200-2

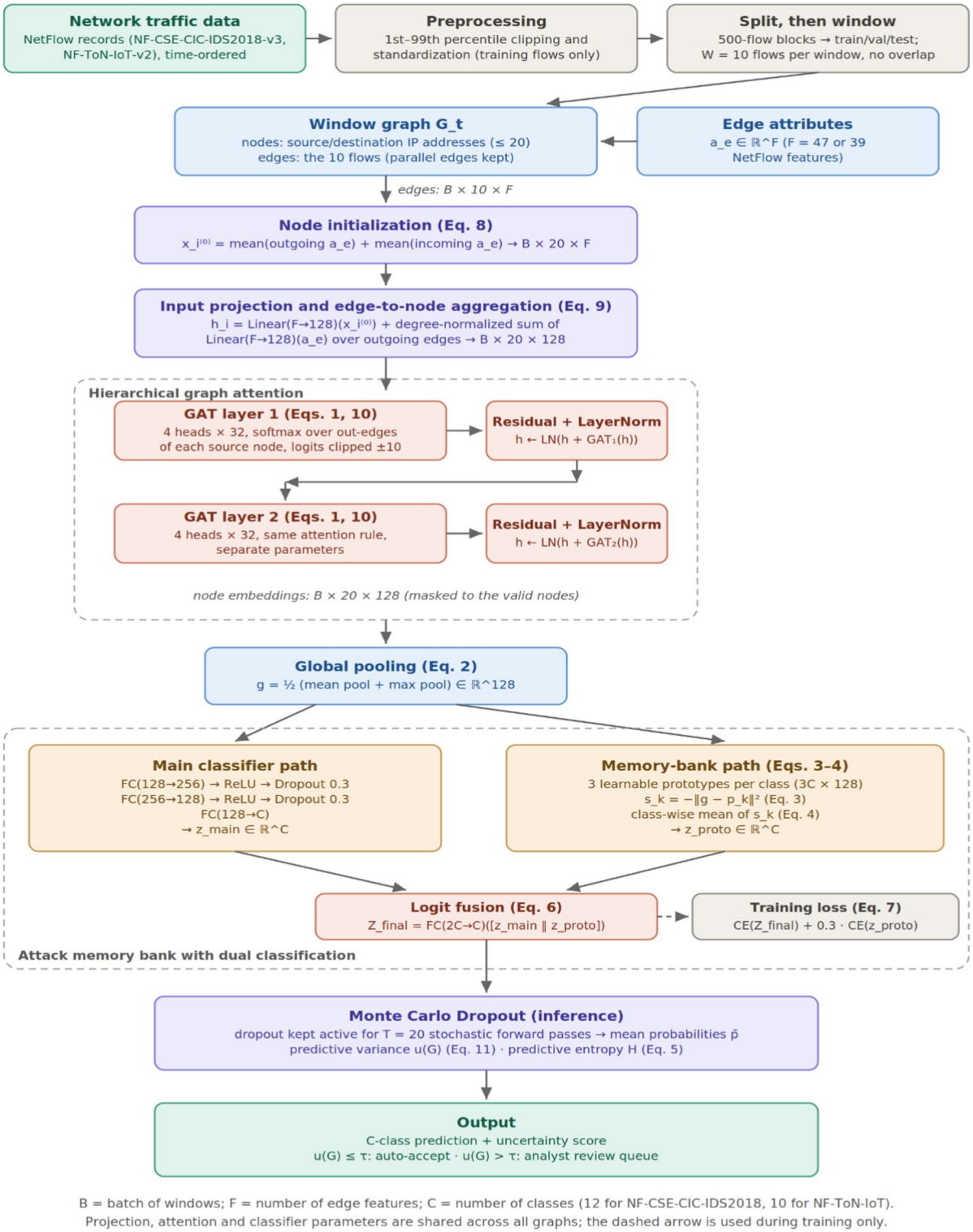


Fig. 1 STAR-IDS architecture. A window of ten flows forms a graph with IP-address nodes; node initialization and edge-to-node aggregation (Eqs. 8–9) feed two graph-attention layers with residual connections and layer normalization (Eqs. 1, 10); the pooled graph embedding (Eq. 2) is classified by a main classifier path and a prototype memory-bank path (Eqs. 3–4) whose logits are fused (Eq. 6); MC-Dropout provides the uncertainty score used for triage (Eqs. 5, 11). B = batch size, F = number of edge features, C = number of classes.

Table 1 NetFlow edge-feature groups used in graph construction (NetFlow v2/v3 feature set; identifiers excluded).

Feature group	Features	Count
Protocol information	PROTOCOL, L7_PROTO	2
Volume (bytes and packets)	IN_BYTES, IN_PKTS, OUT_BYTES, OUT_PKTS, RETRANSMITTED_IN_BYTES, RETRANSMITTED_IN_PKTS, RETRANSMITTED_OUT_BYTES, RETRANSMITTED_OUT_PKTS, NUM_PKTS_UP_TO_128_BYTES, NUM_PKTS_128_TO_256_BYTES, NUM_PKTS_256_TO_512_BYTES, NUM_PKTS_512_TO_1024_BYTES, NUM_PKTS_1024_TO_1514_BYTES	13
Duration and throughput	FLOW_DURATION_MILLISECONDS, DURATION_IN, DURATION_OUT, SRC_TO_DST_SECOND_BYTES, DST_TO_SRC_SECOND_BYTES, SRC_TO_DST_AVG_THROUGHPUT, DST_TO_SRC_AVG_THROUGHPUT	7
TCP flags and windows	TCP_FLAGS, CLIENT_TCP_FLAGS, SERVER_TCP_FLAGS, TCP_WIN_MAX_IN, TCP_WIN_MAX_OUT	5
Packet sizes and TTL	MIN_TTL, MAX_TTL, LONGEST_FLOW_PKT, SHORTEST_FLOW_PKT, MIN_IP_PKT_LEN, MAX_IP_PKT_LEN	6
Application-layer fields (ICMP, DNS, FTP)	ICMP_TYPE, ICMP_IPV4_TYPE, DNS_QUERY_ID, DNS_QUERY_TYPE, DNS_TTL_ANSWER, FTP_COMMAND_RET_CODE	6
Inter-arrival times (v3 only)	SRC_TO_DST_IAT_MIN, SRC_TO_DST_IAT_MAX, SRC_TO_DST_IAT_AVG, SRC_TO_DST_IAT_STDDEV, DST_TO_SRC_IAT_MIN, DST_TO_SRC_IAT_MAX, DST_TO_SRC_IAT_AVG, DST_TO_SRC_IAT_STDDEV	8
Total (NF-CSE-CIC-IDS2018-v3/NF-ToN-IoT-v2)		47/39

rarest class: the still-unassigned blocks containing a class are shuffled and split 70/15/15. Clipping at the 1st and 99th percentiles and standardization are fitted on the training flows only and applied unchanged to validation and test flows, separately in every split and cross-validation fold.

Hierarchical attention mechanism

The hierarchical attention mechanism in STAR-IDS functions at two different levels to identify both fine-grained and coarse-grained patterns of attacks. This is because network attacks have multi-scale properties, where some attacks can be identified based on local anomalies in particular neighborhoods of nodes, while others can be identified based on global graph structures.

Node-level Attention Layer: At the first level, we use a multi-head graph attention mechanism to learn node representations that are discriminative. For each node i, it computes attention weights with its neighbors using Eq. (1).

$$\alpha_{i,j}^{h} = \frac{e^{(LeakyRELU(a^h.[W^h x_i || W^h x_j]))}}{\sum_{k \epsilon N(i)} e^{(LeakyRELU(a^h.[W^h x_i || W^h x_k]))}} \quad (1)$$

where W^h is a learnable weight matrix for head h, a^h is an attention vector, || denotes concatenation, and N(i) represents the neighborhood of node i. The application of LeakyReLU with negative slope 0.2 prevents saturation and allows for the learning of complex attention patterns.

The model implemented with 4 attention heads to effectively explore different aspects of node interactions. The outputs of multiple heads are concatenated and projected to the hidden dimension: x_i^'=Concat (h1, h2, h3, h4), where hk represents the output of the k-th attention head.

To improve gradient flow and facilitate training, we add residual connections and layer normalization: x_i"= LayerNorm(x_i+GAT(x_i, E)), where GAT is the first graph attention layer. This design is further repeated for a second layer, achieving hierarchical feature refinement.

Graph-level Attention and Pooling: After acquiring refined node representations, we transform information to the graph level by employing a combination of global mean pooling and global max pooling:

$$h_{graph} = \frac{1}{2}\left(\frac{1}{|V|}\sum_{i\in V} x_i'' + \underset{i\in V}{max} x_i''\right) \tag{2}$$

This two-pooling approach can capture both the average behavior patterns and the prominent anomalous characteristics, thus forming a comprehensive graph representation. The average pooling part can detect the stable attack signatures, and the max pooling part is able to detect the extreme anomalies that are typical of attacks.

Attack memory bank

A further component of STAR-IDS is the Attack Memory Bank, a learnable module that maintains prototype representations of the classes.

The Attack Memory Bank is a solution to the problem of having a limited number of training samples for some attack categories and is intended to support classes with few training samples; Section "Controlled comparison on NF-CSE-CIC-IDS2018" shows that it does not yield a measurable improvement in our experiments.

The memory bank is represented by K learnable prototype vectors {p1,p2,…,pK}, where K=C×M with C the number of classes (12 for NF-CSE-CIC-IDS2018 and 10 for NF-ToN-IoT, including benign traffic) and M=3, the number of prototypes per class. The prototype vector p_k *R^d has the same dimension as the graph embedding.

The similarity scores between the input graph embedding h_graph and all prototypes are calculated using the Euclidean distance with Eq. (3).

$$s_k = -\|h_{graph} - p_k\|_2^2 \tag{3}$$

The negative squared distance serves as a similarity measure the smaller distances, this indicate higher similarity. These similarity scores are then organized by class and averaged to produce prototype-based logits with Eq. (4).

$$z_{proto} = [\overline{s_1}, \overline{s_2}, \ldots\ldots, \overline{s_C}] \tag{4}$$

where $s_C = \frac{1}{M}\sum_{k\in class} s_k$ represents the average similarity to prototypes of class c.

The prototypes of the memory bank are learned together with the primary network using backpropagation. During training, the prototypes move to represent areas in the feature space where typical attack patterns are found. Prototype matching was designed to provide interpretable class representatives and to generalize to attack variants; these properties were not evaluated in this study.

Bayesian uncertainty quantification

In security-critical systems, knowing how uncertain the model is about a prediction is as important as the prediction itself. In STAR-IDS, Bayesian uncertainty estimation is used to give confidence measures for each detection decision, which helps security analysts to prioritize their responses and allocate their investigation resources accordingly. This model uses Monte Carlo Dropout as an approximation to Bayesian inference. In this, dropout layers with dropout rate 0.3 are added to the classifier network and are kept active during prediction. By doing

forward passes (usually 10–20) with different dropout masks, we get a distribution of predictions instead of a single prediction. For a given input graph, we generate T stochastic predictions {}. The final prediction is obtained by averaging: , and the predictive uncertainty is quantified as the variance of the predicted class probabilities across the T stochastic forward passes, summed over classes. This mechanism was implemented and verified directly: unit tests confirm it produces non-degenerate, input-dependent uncertainty estimates (varying meaningfully across inputs, not a constant value), and the threshold ablation reported in Section "Results and analysis" confirms it has genuine operational value, not merely mathematical validity. $\hat{y}_1, \hat{y}_2, \ldots\ldots\ldots\ldots \hat{y}_T \bar{y} = \frac{1}{T}\sum_{t=1}^{T} \bar{y}_t$

$$H = -\sum_{c=1}^{C} \overline{y_c} log \overline{y_c} \tag{5}$$

High entropy corresponds to high uncertainty, which implies that the sample contains properties that are not strongly linked to a particular attack class. This could be a sign of new attack types or boundary samples that need to be carefully examined by a human expert.

Fusion Architecture: The final classification result combines both the standard classifier path and the memory bank path using a learned fusion module with Eq. (6).

$$Z_{final} = W_{fusion}\left[z_{main} || z_{proto}\right] \tag{6}$$

where z_{main} comes from the main classifier network, z_{proto} from the memory bank, and W_{fusion} is a learnable fusion weight matrix. This architecture allows the model to leverage both learned discriminative features and similarity-based matching for robust attack classification.

The complete STAR-IDS model is trained end-to-end using the combined loss function of Eq. (7):

$$L = L_{main} + \lambda L_{proto} \tag{7}$$

where L_{main} and L_{proto} are cross-entropy losses for the main classifier and prototype classifier respectively, and $\lambda = 0.3$ balances their contributions. This multi-objective optimization ensures that both pathways learn complementary representations, enhancing overall detection performance.

Implementation-level definition of the graph operations

For reproducibility, the operations are defined here exactly as implemented. Let ε_i^{out} and ε_i^{in} be the outgoing and incoming edges of node i, s(e) and d(e) the source and destination of edge e, and $a_e \in \mathbb{R}^F$ its standardized feature vector. Node features are initialized by Eq. (8) (a term is omitted when the node has no edges of that direction). The node and edge projections (Linear $F \rightarrow 128$, parameters shared across all graphs) and the degree-normalized edge-to-node aggregation give the initial hidden states of Eq. (9). Each of the two graph-attention layers has four heads of dimension 32; the attention of an edge is normalized over the outgoing edges of its source node and messages flow from destination to source (Eq. (10)). Parallel edges are treated as separate edges. Each layer is followed by a residual connection and layer normalization, and the graph embedding is the average of global mean and max pooling over the valid nodes (Eq. (2)). Prototypes are initialized from a standard normal distribution. Window tensors have shape $B \times 10 \times F$ for edges and $B \times 20 \times 128$ for nodes (padded, with a node mask). The implementation uses PyTorch 2.6.0 without a graph-learning library. For MC-Dropout, dropout remains active during $T = 20$ forward passes; the review decision uses the summed predictive variance of Eq. (11), and the predictive entropy of Eq. (5) is reported as a second uncertainty score.

$$x_i^{(0)} = \frac{1}{|\mathcal{E}_i^{out}|}\sum_{e \in \mathcal{E}_i^{out}} \mathbf{a}_e + \frac{1}{|\mathcal{E}_i^{in}|}\sum_{e \in \mathcal{E}_i^{in}} \mathbf{a}_e \tag{8}$$

$$h_i = W_n x_i^{(0)} + b_n + \frac{1}{\max\left(1, |\mathcal{E}_i^{out}|\right)} \sum_{e \in \mathcal{E}_i^{out}} \left(W_e \mathbf{a}_e + b_e\right) \tag{9}$$

$$\alpha_e^{(k)} = \frac{\exp\left(\mathrm{clip}\left(\mathrm{LeakyReLU}\left(\mathbf{a}^{(k)\top}\left[W^{(k)} h_{s(e)} \,\|\, W^{(k)} h_{d(e)}\right]\right), -10, 10\right)\right)}{\sum_{e' \in \mathcal{E}_{s(e)}^{out}} \exp\left(\cdot\right)}, h_i' = \|_{k=1}^{4} \sum_{e \in \mathcal{E}_i^{out}} \alpha_e^{(k)} W^{(k)} h_{d(e)} \tag{10}$$

$$u\left(G\right) = \sum_{c=1}^{C} \frac{1}{T-1} \sum_{t=1}^{T} \left(p_t\left(c \mid G\right) - \overline{p}\left(c \mid G\right)\right)^2 \tag{11}$$

Experimental setup

Datasets and sampling

Two NetFlow datasets with endpoint identifiers were used[9]. The main dataset is NF-CSE-CIC-IDS2018-v3[10] (20,115,529 flows, whose flows carry real IP addresses and flow start timestamps,flows were ordered by FLOW_START_MILLISECONDS. The second dataset is the IoT dataset NF-ToN-IoT-v2 (13,135,881 flows), used to assess generalization to IoT/IIoT traffic. The available release of NF-ToN-IoT-v2 contains neither IP addresses nor timestamps, so its flows were kept in file order and its graphs use the synthetic node mapping of Section "Dynamic graph construction"; topology-related conclusions are therefore drawn from NF-CSE-CIC-IDS2018 only. Because of the dataset sizes, at most 600,000 benign flows and 150,000 flows per attack class were kept; each flow of a class was retained with probability min(1, cap/count), which preserves the order of the retained flows. Classes with fewer than 2,000 retained flows were removed (the three web attacks of NF-CSE-CIC-IDS2018: Brute Force-Web, Brute Force-XSS and SQL Injection). Table 2 lists raw counts, sampling fractions and retained counts, which total 1,805,755 and 1,526,254 flows.

Graphs per class and split

Tables 3 and 4 give the number of graphs per class in each split. NF-CSE-CIC-IDS2018 yields 180,575 graphs (126,300/27,150/27,125 for training, validation and test) and NF-ToN-IoT 152,625 graphs (106,925/22,850/22,850). With real IP nodes, a graph of NF-CSE-CIC-IDS2018 has on average 7.7 nodes (2–20), 92.7% of the graphs contain parallel edges, and the mean window purity (share of the most frequent class) is 0.929, with 65.6% single-class windows, reflecting the temporal clustering of attack flows. On NF-ToN-IoT the synthetic mapping yields only 3.2 nodes per graph on average.

Evaluation protocol

The main protocol is the block-wise hold-out of Section "Dynamic graph construction" (70/15/15 of the 500-flow blocks of each class). Temporally blocked 5-fold cross-validation assigns whole blocks to folds in the same class-wise manner; in each fold, 85% of the remaining blocks are used for training and 15% for validation. Because each attack type of CSE-CIC-IDS2018 occurs only on particular days, a strict forward-in-time split leaves classes unseen; it is therefore reported as a control (Section "Effect of the evaluation protocol and of the graph semantics"), together with a protocol in which rows are shuffled before windowing, a common choice when no timestamps are available. For every configuration, five runs with seeds 42–46 were made on the identical split; runs are paired by seed (same split, initialization seed and batch order). For each comparison with the full model, we report the mean paired difference with its 95% confidence interval (t distribution, n = 5), the paired t-test and the Wilcoxon signed-rank test, with Holm correction over all comparisons of a dataset. With five pairs, small

Table 2 Raw and retained class counts and sampling fractions.

Dataset	Class	Raw flows	Sampling fraction	Retained flows	Used
NF-CSE-CIC-IDS2018-v3	Benign	17,514,626	0.0343	599,560	Yes
NF-CSE-CIC-IDS2018-v3	DDOS_attack-HOIC	1,032,311	0.1453	150,060	Yes
NF-CSE-CIC-IDS2018-v3	FTP-BruteForce	386,720	0.3879	149,622	Yes
NF-CSE-CIC-IDS2018-v3	DDoS_attacks-LOIC-HTTP	288,589	0.5198	150,087	Yes
NF-CSE-CIC-IDS2018-v3	Bot	207,703	0.7222	150,252	Yes
NF-CSE-CIC-IDS2018-v3	SSH-Bruteforce	188,474	0.7959	149,910	Yes
NF-CSE-CIC-IDS2018-v3	Infilteration	188,152	0.7972	149,848	Yes
NF-CSE-CIC-IDS2018-v3	DoS_attacks-SlowHTTPTest	105,550	1.0000	105,550	Yes
NF-CSE-CIC-IDS2018-v3	DoS_attacks-Hulk	100,076	1.0000	100,076	Yes
NF-CSE-CIC-IDS2018-v3	DoS_attacks-GoldenEye	61,300	1.0000	61,300	Yes
NF-CSE-CIC-IDS2018-v3	DoS_attacks-Slowloris	36,040	1.0000	36,040	Yes
NF-CSE-CIC-IDS2018-v3	DDOS_attack-LOIC-UDP	3,450	1.0000	3,450	Yes
NF-CSE-CIC-IDS2018-v3	Brute_Force_-Web	1,618	1.0000	1,618	No
NF-CSE-CIC-IDS2018-v3	Brute_Force_-XSS	480	1.0000	480	No
NF-CSE-CIC-IDS2018-v3	SQL_Injection	440	1.0000	440	No
NF-ToN-IoT-v2	Benign	3,601,284	0.1666	599,762	Yes
NF-ToN-IoT-v2	Scanning	3,002,169	0.0500	149,722	Yes
NF-ToN-IoT-v2	xss	2,449,955	0.0612	150,141	Yes
NF-ToN-IoT-v2	ddos	1,746,590	0.0859	150,055	Yes
NF-ToN-IoT-v2	Password	993,718	0.1510	149,647	Yes
NF-ToN-IoT-v2	Injection	660,467	0.2271	149,924	Yes
NF-ToN-IoT-v2	dos	654,359	0.2292	149,664	Yes
NF-ToN-IoT-v2	Backdoor	16,259	1.0000	16,259	Yes
NF-ToN-IoT-v2	mitm	7,723	1.0000	7,723	Yes
NF-ToN-IoT-v2	Ransomware	3,357	1.0000	3,357	Yes

Table 3 Graphs per class and split, NF-CSE-CIC-IDS2018-v3 (block-wise hold-out).

Class	Train	Val	Test
Benign	29,741	6,353	6,419
Bot	13,358	2,897	2,900
DDOS_attack-HOIC	10,902	2,350	2,334
DDOS_attack-LOIC-UDP	401	79	79
DDoS_attacks-LOIC-HTTP	11,057	2,367	2,272
DoS_attacks-GoldenEye	4,403	947	941
DoS_attacks-Hulk	7,059	1,521	1,550
DoS_attacks-SlowHTTPTest	7,798	1,700	1,659
DoS_attacks-Slowloris	2,860	600	600
FTP-BruteForce	11,249	2,400	2,400
Infilteration	16,384	3,536	3,571
SSH-Bruteforce	11,088	2,400	2,400
Total graphs	126,300	27,150	27,125
Flows	1,263,000	271,500	271,250

differences cannot be detected reliably; the confidence intervals indicate the range of effects compatible with the data. Results are reported as mean ± standard deviation over runs, with 95% confidence intervals of the mean where stated. All models were trained with the same budget (at most 30 epochs, early stopping on the validation loss with patience 10); early stopping and model selection never use test data.

Table 4 Graphs per class and split, NF-ToN-IoT-v2 (block-wise hold-out).

Class	Train	Val	Test
Benign	30,338	6,017	6,763
Backdoor	1,608	328	332
ddos	5,366	1,319	865
dos	16,305	3,047	3,360
Injection	19,776	4,623	4,339
mitm	2,889	619	551
Password	9,470	2,325	1,543
Ransomware	267	78	118
Scanning	11,501	2,576	2,491
xss	9,405	1,918	2,488
Total graphs	106,925	22,850	22,850
Flows	1,069,250	228,500	228,500

Table 5 Hyperparameters (identical for STAR-IDS, ablations and baselines where applicable).

Hyperparameter	Value
Window/stride	10 flows/10 flows (no overlap)
Block size	500 flows (50 windows)
Hidden dimension; attention heads	128; 4 heads × 32 per GAT layer
GAT layers	2 (residual + LayerNorm)
Classifier	128 → 256 → 128 → C, ReLU, dropout 0.3
Memory bank	3 prototypes per class, $\lambda = 0.3$
Optimizer	AdamW, learning rate 10^{-3}, weight decay 10^{-4}
Scheduler	ReduceLROnPlateau (factor 0.5, patience 5)
Batch size	256 graphs
Epochs; early stopping	at most 30; patience 10 on validation loss
Gradient clipping	L2 norm 1.0
Loss	class-weighted cross-entropy
Clipping/scaling	1st–99th percentile, StandardScaler (training flows only)
MC-Dropout samples	20
Seeds	42, 43, 44, 45, 46

Controlled baselines

All baselines use the same windows, splits, features, pooling, classifier head, class-weighted loss, optimizer and training budget as STAR-IDS: E-GraphSAGE[8], which aggregates incident edge features and classifies from edge embeddings $[h_u \,\|\, h_v]$; GAT[44] with self-loops, GCN[45], GraphSAGE with mean aggregation[46],and a flow-set MLP that receives the same ten flows without any graph structure. The ablations remove the second attention layer, the memory bank, or both.

Implementation, hyperparameters and environment

Table 5 lists the hyperparameters, which are identical for STAR-IDS, its ablations and the baselines wherever they apply, and Table 6 reports the execution environment used for all experiments and latency measurements.

Table 6 Execution environment (Kaggle).

Item	Value
GPU	Tesla T4
GPU memory (GB)	15.6
NVIDIA driver	580.178.04 580.178.04
CUDA runtime/cuDNN	12.4/90,300
CPU	Intel(R) Xeon(R) CPU @ 2.00 GHz
CPU cores/RAM (GB)	4/32.9
Python	3.11.13
PyTorch	2.6.0+cu124
NumPy/pandas	1.26.4/2.2.3
scikit-learn/SciPy	1.2.2/1.15.3
Precision	float32

Table 7 Block-wise hold-out results on NF-CSE-CIC-IDS2018 (mean ± SD over five paired seeds; 95% CI of the mean).

Configuration	Accuracy (%)	95% CI±	Macro-F1 (%)	95% CI±	MCC	Macro AUC	Best epoch
STAR-IDS (full)	87.61 ± 1.22	1.52	86.41 ± 0.91	1.13	0.862	0.991	28.0
w/o hierarchical attention	86.56 ± 0.63	0.78	86.77 ± 0.84	1.04	0.850	0.991	26.6
w/o memory bank	87.18 ± 0.28	0.35	87.43 ± 0.36	0.45	0.856	0.991	24.4
w/o both	87.54 ± 1.17	1.45	86.05 ± 0.71	0.88	0.861	0.991	20.0
E-GraphSAGE	87.40 ± 0.37	0.46	87.62 ± 0.53	0.66	0.859	0.991	22.6
GAT	87.53 ± 0.84	1.04	87.17 ± 0.54	0.67	0.861	0.991	25.4
GCN	87.56 ± 0.83	1.03	87.18 ± 0.41	0.52	0.861	0.991	24.6
GraphSAGE	87.07 ± 0.28	0.35	87.22 ± 0.53	0.65	0.856	0.991	20.0
MLP (flow set, no graph)	87.08 ± 0.22	0.27	86.84 ± 1.16	1.44	0.855	0.991	23.8

Results and analysis

Controlled comparison on NF-CSE-CIC-IDS2018

Table 7 reports the hold-out results. STAR-IDS reached 87.61 ± 1.22% accuracy and 86.41 ± 0.91% macro-F1 (mean ± SD over five seeds). All other configurations lay in the same range: E-GraphSAGE 87.40 ± 0.37% accuracy and 87.62 ± 0.53% macro-F1, GAT, GCN and GraphSAGE between 87.07% and 87.56% accuracy, and the graph-free MLP 87.08 ± 0.22%. None of the 16 paired comparisons with the full model was significant after Holm correction (Table 8). The largest difference was the 1.21-point higher macro-F1 of E-GraphSAGE (95% CI 0.13–2.29 points; unadjusted paired t-test $p = 0.036$, Holm-adjusted $p = 0.57$). Neither the second attention layer nor the memory bank produced a detectable improvement, and the comparable performance of the MLP shows that, on this task, the communication topology within a window adds little information beyond the flow features themselves. Figure 2 summarizes the results.

Per-class results

Per-class results of the full model (seed 42) are given in Table 9 and Fig. 3. Bot, DDoS-HOIC, DoS-GoldenEye, DoS-Hulk, DoS-Slowloris and SSH-Bruteforce reach F1-scores above 99%, whereas DoS-SlowHTTPTest is almost never detected (recall 1.9%) and FTP-BruteForce has a precision of only 59.4%; the confusion matrix shows that 1,628 of the 1,659 SlowHTTPTest test windows are predicted as FTP-BruteForce, indicating that the NetFlow features of the two classes are almost indistinguishable in this dataset. Infiltration (F1 79.4%) and benign traffic (F1 88.1%) are mainly confused with each other (655 Infiltration windows were predicted as benign and 844 benign windows as Infiltration).

Table 8 Paired comparisons with the full model on NF-CSE-CIC-IDS2018 (difference = full − other, percentage points; n = 5 pairs).

Comparison	Metric	Mean difference	95% CI	Paired t p	Wilcoxon p	Holm p (t)
STAR-IDS − w/o hierarchical attention	accuracy	+1.06	−1.06 to +3.17	0.238	0.188	1.000
STAR-IDS − w/o hierarchical attention	macro-F1	−0.36	−0.94 to +0.22	0.161	0.312	1.000
STAR-IDS − w/o memory bank	accuracy	+0.44	−1.03 to +1.90	0.454	0.625	1.000
STAR-IDS − w/o memory bank	macro-F1	−1.02	−2.21 to +0.17	0.075	0.125	1.000
STAR-IDS − w/o both	accuracy	+0.07	−1.60 to +1.75	0.911	1.000	1.000
STAR-IDS − w/o both	macro-F1	+0.35	−0.91 to +1.62	0.482	0.438	1.000
STAR-IDS − E-GraphSAGE	accuracy	+0.21	−1.21 to +1.63	0.700	0.812	1.000
STAR-IDS − E-GraphSAGE	macro-F1	−1.21	−2.29 to −0.13	0.036	0.125	0.570
STAR-IDS − GAT	accuracy	+0.08	−1.01 to +1.17	0.849	0.812	1.000
STAR-IDS − GAT	macro-F1	−0.76	−2.43 to +0.91	0.274	0.438	1.000
STAR-IDS − GCN	accuracy	+0.06	−1.16 to +1.28	0.902	0.812	1.000
STAR-IDS − GCN	macro-F1	−0.78	−2.32 to +0.77	0.235	0.312	1.000
STAR-IDS − GraphSAGE	accuracy	+0.54	−0.93 to +2.01	0.365	0.625	1.000
STAR-IDS − GraphSAGE	macro-F1	−0.81	−2.49 to +0.86	0.248	0.312	1.000
STAR-IDS − MLP (flow set, no graph)	accuracy	+0.54	−0.79 to +1.86	0.324	0.438	1.000
STAR-IDS − MLP (flow set, no graph)	macro-F1	−0.43	−2.70 to +1.84	0.627	0.812	1.000

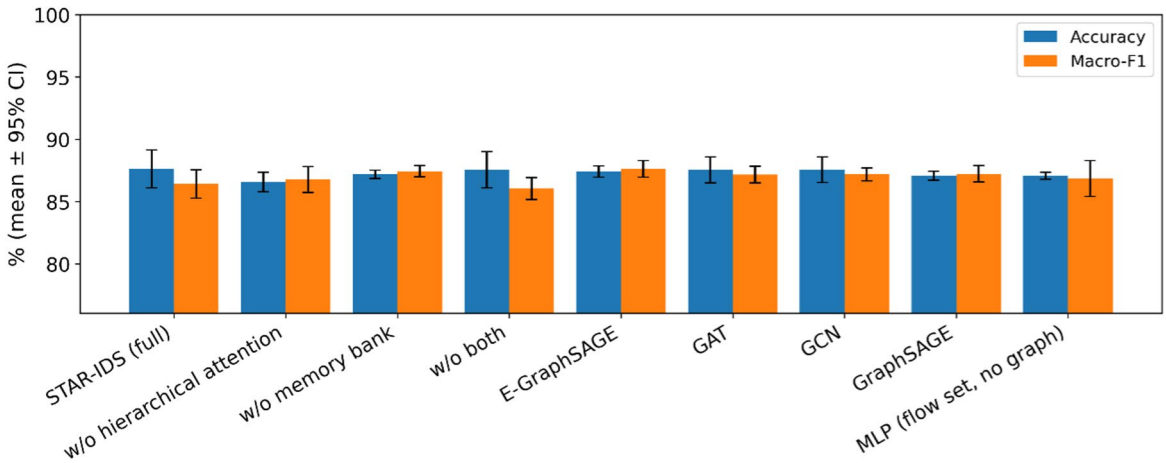


Fig. 2 Accuracy and macro-F1 (mean ± 95% CI over five seeds) of STAR-IDS, its ablations and the baselines on NF-CSE-CIC-IDS2018.

Table 9 Per-class results of STAR-IDS on NF-CSE-CIC-IDS2018 (block-wise hold-out, seed 42).

Class	Test graphs	Precision (%)	Recall (%)	F1 (%)
Benign	6,419	89.45	86.74	88.07
Bot	2,900	99.93	100.00	99.97
DDOS_attack-HOIC	2,334	99.91	99.96	99.94
DDOS_attack-LOIC-UDP	79	72.48	100.00	84.04
DDoS_attacks-LOIC-HTTP	2,272	100.00	98.50	99.25
DoS_attacks-GoldenEye	941	99.58	99.89	99.73
DoS_attacks-Hulk	1,550	100.00	99.74	99.87
DoS_attacks-SlowHTTPTest	1,659	64.58	1.87	3.63
DoS_attacks-Slowloris	600	98.20	100.00	99.09
FTP-BruteForce	2,400	59.38	99.29	74.32
Infilteration	3,571	77.44	81.41	79.37
SSH-Bruteforce	2,400	100.00	100.00	100.00

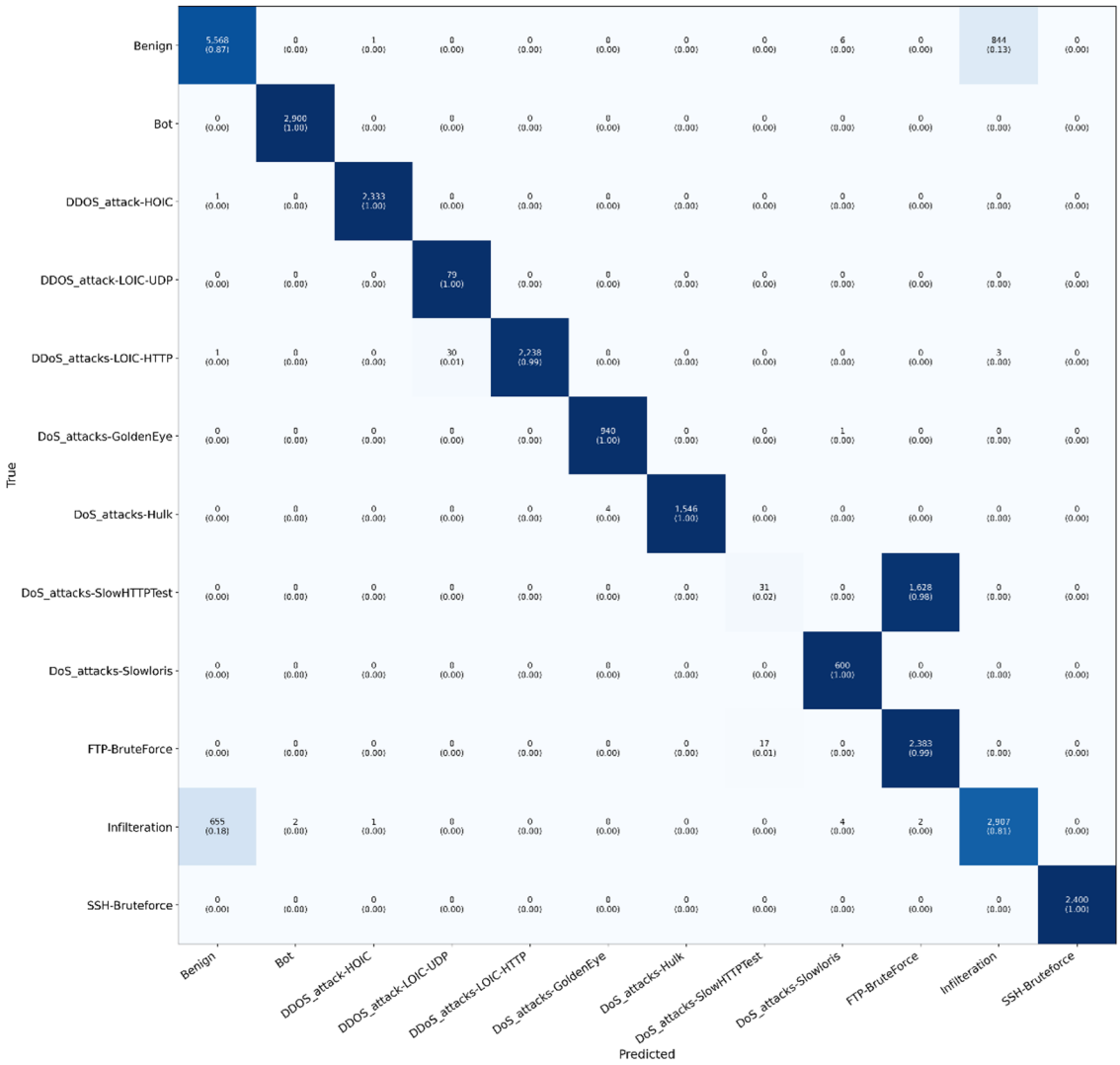


Fig. 3 Confusion matrix of STAR-IDS on NF-CSE-CIC-IDS2018 (counts and row-normalized rates; seed 42).

Generalization to IoT traffic: NF-ToN-IoT

On NF-ToN-IoT (Table 10 and Fig. 4), STAR-IDS reached 77.21±3.11% accuracy and 75.62±1.70% macro-F1. The graph-free MLP obtained the highest scores (81.10±1.55% and 78.82±0.71%), followed by GAT (79.63±1.39%). The full model was 3.89 points less accurate than the MLP (95% CI −6.57 to −1.21; unadjusted $p=0.016$, Holm-adjusted $p=0.24$); no comparison remained significant after correction. Because this release lacks IP addresses, these graphs carry no host topology, and the result mainly shows that the STAR-IDS components do not provide an advantage on IoT traffic in this setting. The hardest classes were man-in-the-middle (F1 26.9%), DDoS, DoS and injection.

Temporally blocked cross-validation

Blocked 5-fold cross-validation (Table 11) confirms the hold-out results. On NF-CSE-CIC-IDS2018, STAR-IDS reached 86.70%±0.73% accuracy, E-GraphSAGE 87.43%, GAT 87.21% and the MLP 87.39%; on NF-ToN-IoT

Table 10 Block-wise hold-out results on NF-ToN-IoT (mean ± SD over five paired seeds; synthetic node mapping).

Configuration	Accuracy (%)	95% CI±	Macro-F1 (%)	95% CI±	MCC	Macro AUC	Best epoch
STAR-IDS (full)	77.21 ± 3.11	3.86	75.62 ± 1.70	2.11	0.737	0.980	28.4
w/o hierarchical attention	77.10 ± 3.13	3.89	75.42 ± 1.66	2.06	0.735	0.980	28.0
w/o memory bank	78.59 ± 1.00	1.25	76.48 ± 0.51	0.64	0.750	0.981	16.4
w/o both	78.73 ± 1.75	2.18	76.55 ± 0.89	1.11	0.752	0.981	18.6
E-GraphSAGE	77.88 ± 2.11	2.61	76.14 ± 1.42	1.76	0.744	0.981	13.8
GAT	79.63 ± 1.39	1.72	77.39 ± 0.81	1.01	0.762	0.981	23.4
GCN	76.71 ± 2.45	3.05	75.46 ± 1.75	2.17	0.731	0.981	18.6
GraphSAGE	77.79 ± 2.40	2.98	75.91 ± 1.50	1.86	0.743	0.981	13.6
MLP (flow set, no graph)	81.10 ± 1.55	1.92	78.82 ± 0.71	0.88	0.779	0.983	22.2

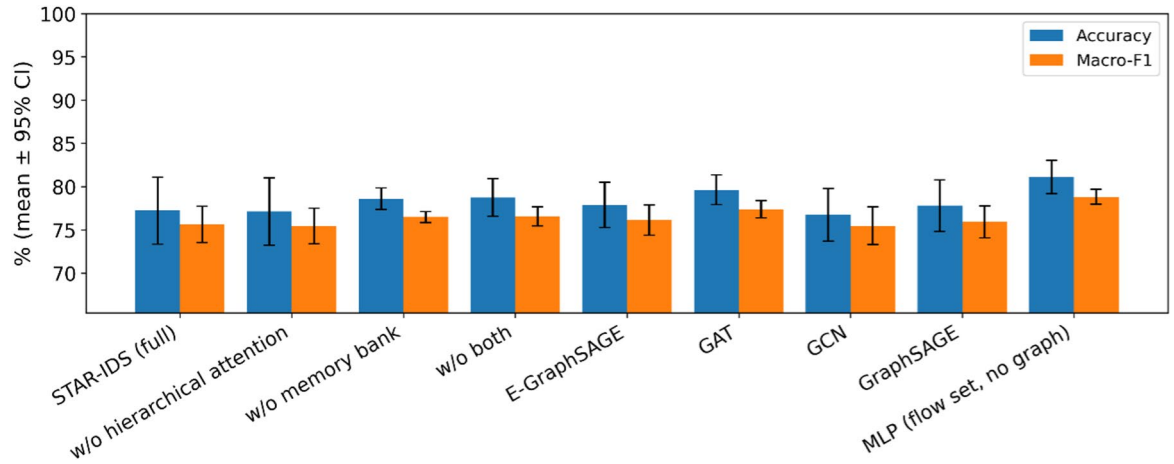


Fig. 4 Accuracy and macro-F1 (mean ± 95% CI over five seeds) on NF-ToN-IoT.

Table 11 Temporally blocked 5-fold cross-validation (mean ± SD over folds; Δ = other − STAR-IDS, paired by fold).

Dataset	Model	Accuracy (%)	Macro-F1 (%)	MCC	Δ macro-F1 versus STAR-IDS (95% CI)	Holm p
NF-CSE-CIC-IDS2018	STAR-IDS (full)	86.70 ± 0.73	85.90 ± 1.54	0.852	–	–
NF-CSE-CIC-IDS2018	E-GraphSAGE	87.43 ± 1.03	87.46 ± 1.03	0.859	+1.56 (−0.35 to +3.46)	0.26
NF-CSE-CIC-IDS2018	GAT	87.21 ± 0.82	86.78 ± 1.93	0.857	+0.88 (−0.21 to +1.97)	0.26
NF-CSE-CIC-IDS2018	MLP (flow set, no graph)	87.39 ± 0.38	86.09 ± 1.94	0.859	+0.19 (−1.54 to +1.91)	0.78
NF-ToN-IoT	STAR-IDS (full)	81.74 ± 3.90	77.92 ± 2.82	0.787	–	–
NF-ToN-IoT	E-GraphSAGE	82.18 ± 2.35	78.95 ± 3.33	0.793	+1.03 (−2.69 to +4.76)	0.97
NF-ToN-IoT	GAT	81.11 ± 2.49	77.60 ± 3.59	0.781	−0.32 (−4.15 to +3.51)	0.97
NF-ToN-IoT	MLP (flow set, no graph)	83.18 ± 1.95	79.45 ± 3.10	0.805	+1.54 (−1.66 to +4.74)	0.76

the corresponding values were 81.74%, 82.18%, 81.11% and 83.18%. None of the fold-paired macro-F1 differences was significant (Holm-adjusted $p \geq 0.26$).

Effect of the evaluation protocol and of the graph semantics

Table 12 quantifies how the evaluation choices common in existing work change the results (seed 42, otherwise identical settings). When rows were shuffled before windowing, a common choice when timestamps are unavailable, accuracy rose to 90.80%, while macro-F1 fell to 65.05%, because windows then mix flows from the whole capture and minority classes are swamped. Under a strict forward-in-time split, the test period contains classes never seen in training (Bot and Infiltration) and accuracy dropped to 22.28%, so results obtained with shuffled or randomly split data say little about deployment on future traffic. Replacing the real IP nodes with the synthetic

Table 12 Effect of the evaluation protocol and of the node mapping (seed 42).

Dataset	Control	Model	Accuracy (%)	Macro-F1 (%)	MCC
NF-CSE-CIC-IDS2018	Rows shuffled before windowing	STAR-IDS (full)	90.80	65.05	0.891
NF-CSE-CIC-IDS2018	Rows shuffled before windowing	MLP (flow set, no graph)	86.00	61.25	0.833
NF-CSE-CIC-IDS2018	Strict forward-in-time split (first 70% train, last 15% test); unseen test classes: Bot, Infilteration	STAR-IDS (full)	22.28	6.49	0.222
NF-CSE-CIC-IDS2018	Strict forward-in-time split (first 70% train, last 15% test); unseen test classes: Bot, Infilteration	MLP (flow set, no graph)	23.48	5.85	0.189
NF-CSE-CIC-IDS2018	Synthetic (f × 1000 mod 150) nodes, same blocked split	STAR-IDS (full)	86.44	84.52	0.849
NF-CSE-CIC-IDS2018	Reference: real nodes, blocked split (main protocol)	STAR-IDS (full)	88.20	85.61	0.869
NF-CSE-CIC-IDS2018	Reference: real nodes, blocked split (main protocol)	MLP (flow set, no graph)	87.32	87.79	0.857
NF-ToN-IoT	Rows shuffled before windowing	STAR-IDS (full)	61.39	39.89	0.481
NF-ToN-IoT	Rows shuffled before windowing	MLP (flow set, no graph)	76.07	47.28	0.658
NF-ToN-IoT	Strict forward-in-time split (first 70% train, last 15% test); unseen test classes: backdoor	STAR-IDS (full)	54.81	28.47	0.432
NF-ToN-IoT	Strict forward-in-time split (first 70% train, last 15% test); unseen test classes: backdoor	MLP (flow set, no graph)	62.74	33.81	0.522
NF-ToN-IoT	Reference: synthetic nodes, blocked split (main protocol)	STAR-IDS (full)	78.58	76.25	0.749
NF-ToN-IoT	Reference: synthetic nodes, blocked split (main protocol)	MLP (flow set, no graph)	81.74	79.61	0.785

Table 13 Node identifiers on NF-CSE-CIC-IDS2018: original synthetic mapping versus real IP addresses.

Node mapping	Distinct raw values	Node identifiers used	Share of values colliding
Synthetic from PROTOCOL	5	3	0.400
Synthetic from L7_PROTO	120	3	0.975
Real source IP	29,791	29,791	0.000
Real destination IP	10,321	10,321	0.000

modulo mapping reduced accuracy from 88.20% to 86.44% and macro-F1 from 85.61% to 84.52%. The synthetic mapping collapses the 5 protocol values and 120 L7_PROTO values of this dataset onto three node identifiers each (Table 13), so it cannot represent hosts. These controls show that the protocol has a larger effect on the reported numbers than any architectural component evaluated here.

Convergence and training budget

All configurations were trained with the same budget. The full model reached its lowest validation loss on average in epoch 28.0 of 30 on NF-CSE-CIC-IDS2018 (baselines 20.0–25.4), and Fig. 5 shows that validation accuracy had levelled off for all configurations, whereas the validation loss of the memory-bank variants was still decreasing slowly. A longer budget might therefore benefit STAR-IDS slightly; with validation accuracy flat over the last epochs, we do not expect this to change the conclusions, but we cannot exclude a small effect.

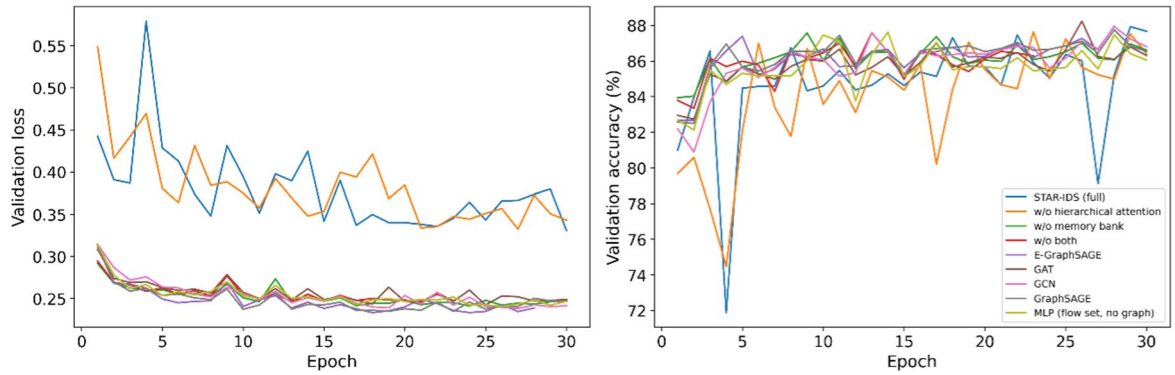


Fig. 5 Validation loss and accuracy per epoch for all configurations on NF-CSE-CIC-IDS2018 (seed 42).

Table 14 Uncertainty threshold analysis (MC-Dropout, T = 20; full model, seed 42).

Dataset	Variance threshold	Flagged (n)	Flagged (%)	Auto-accept accuracy (%)	Review-queue accuracy (%)
NF-CSE-CIC-IDS2018	0.005	378	1.39	88.51	68.25
NF-CSE-CIC-IDS2018	0.01	129	0.48	88.31	71.32
NF-CSE-CIC-IDS2018	0.02	86	0.32	88.29	68.60
NF-CSE-CIC-IDS2018	0.05	60	0.22	88.27	70.00
NF-CSE-CIC-IDS2018	0.1	43	0.16	88.27	62.79
NF-ToN-IoT	0.005	6,936	30.35	85.33	63.11
NF-ToN-IoT	0.01	4,936	21.60	83.82	59.58
NF-ToN-IoT	0.02	2,856	12.50	81.84	55.74
NF-ToN-IoT	0.05	706	3.09	79.39	53.12
NF-ToN-IoT	0.1	120	0.53	78.70	56.67

Bayesian uncertainty

MC-Dropout uncertainty identified erroneous predictions well: the area under the ROC curve for separating wrong from correct predictions was 0.812 for the predictive variance and 0.891 for the entropy on NF-CSE-CIC-IDS2018, and 0.748 and 0.811 on NF-ToN-IoT. With the variance thresholds of Table 14, the flagged predictions were clearly less accurate than the auto-accepted ones (for example 71.3% versus 88.3% at threshold 0.01 on NF-CSE-CIC-IDS2018). Only a small share of predictions was flagged ($\leq$1.4%), so the thresholds can be lowered if more analyst capacity is available.

Inference latency and computational cost

Latency was measured on the hardware of Table 6 (float32, 100 trials after 10 warm-up runs). For a single graph, inference took 2.90 ms on the GPU (95th percentile 3.09 ms) with the input already on the device and 3.06 ms including host-to-device transfer, and 1.91 ms on one CPU thread. In batches of 256 graphs the GPU processed 87,322 graphs per second. Graph construction cost 38 µs per graph, so the complete pipeline (construction plus batched inference) handled about 201,800 flows per second. The model has 118,712 parameters on NF-CSE-CIC-IDS2018 and 115,548 on NF-ToN-IoT.

Table 15 Accuracies reported in recent studies (context only; not comparable with this work).

System	Model	Data Set	Accuracy
Madhloom Kurdi et al.[12]	QDA, LDA, ML	KDD, UNSW	0.90
Ahmed et al.[13]	LSTM	UNSW	0.96
Olanrewaju-George and Pranggono[28]	DNN	Mirai and BASHLITE	0.90
Kumar and Swarnkar[30]	QSVM		0.91
Kamal and Mashaly[27]	DNN	CICIDS2017	0.99
Kamalakkannan et al.[15]	Bi-LSTM	KDD	0.81
Bandarupalli[23]	DNN	CICIDS2017	0.97
Rahman et al.[39]	GAN+SVM	UNSW-NB15	0.98
Shi et al.[34]	hummingbird algorithm	CICIDS2017	0.99
Berguiga et al.[16,17]	HIDS-IoMT	IOTID20	0.99
SaiRamesh et al.[47]	Mario game-based solution	NSL-KDD	0.98
Sudha and Bolla[29]	Puffer fish Federated Learning (PFFL)	NSL-KDD	0.99
Thaljaoui[48]	CNN-LSTM	UNSW-NB15	0.78

Results reported in the literature

Table 15 lists accuracies reported by recent studies for context only. They were obtained on different datasets, splits and preprocessing, mostly with random flow-level splits, and Section "Effect of the evaluation protocol and of the graph semantics" shows that such protocol differences alone can change accuracy by tens of percentage points; the values are therefore not comparable with ours. The controlled comparison of Sections "Controlled comparison on NF-CSE-CIC-IDS2018"–"Temporally blocked cross-validation" is the basis for our conclusions.

Discussion and limitations

The evaluation leads to clear conclusions about STAR-IDS. With real host nodes, chronological windows, leakage-free splits, equal training budgets and paired seeds, STAR-IDS performs on a par with standard GNN intrusion detectors and with a model that ignores the graph structure. Neither the hierarchical attention nor the memory bank yields a statistically detectable benefit, and on IoT traffic the simpler models are at least as good. Advantages measured with shuffled windows or synthetic node identifiers should not be regarded as evidence for a graph architecture, because Section "Effect of the evaluation protocol and of the graph semantics" shows that these choices change the results substantially. What remains supported is the MC-Dropout uncertainty mechanism, which reliably separates error-prone predictions and could be used to route them to analysts, and the leakage-controlled graph construction and evaluation pipeline, which we release so that graph-based IDS can be compared under identical conditions.

The study has several limitations. (1) The IoT dataset release used contains no IP addresses or timestamps, so topology could only be evaluated on NF-CSE-CIC-IDS2018. (2) Sampling caps change the class balance and the density of flows in a window relative to the raw traffic. (3) Three web-attack classes were too small to be included, and DoS-SlowHTTPTest is essentially undetectable from these NetFlow features. (4) Five paired seeds detect only moderately large differences; small effects cannot be excluded. (5) The training budget of 30 epochs may be slightly short for the memory-bank variants. (6) A window of ten flows is a short time slice; larger or time-based windows, which may make topology more informative, were not explored. (7) The forward-in-time control shows that detection of attack types unseen in training remains unsolved.

Conclusion

This paper presented STAR-IDS, which combines dynamic graph construction over windows of network flows, hierarchical graph attention, a prototype memory bank and MC-Dropout uncertainty, and evaluated it under a leakage-controlled protocol on two NetFlow datasets. On NF-CSE-CIC-IDS2018, with real IP-address nodes and chronologically ordered flows, STAR-IDS reached 87.61% accuracy and 86.41% macro-F1, statistically indistinguishable from E-GraphSAGE, GAT, GCN, GraphSAGE and a graph-free MLP; on NF-ToN-IoT, the simpler models performed at least as well. The uncertainty mechanism identified erroneous predictions with an AUC of up to 0.89. The controls show that shuffled windows, synthetic node mappings and forward-in-time splits change the results more than the architectural components do.

For practice and research, the results suggest that graph-based IDS should be evaluated against graph-free and standard GNN baselines under identical, temporally sound protocols with real endpoint identifiers, and that uncertainty estimates are a practical tool for analyst triage. Future work will evaluate time-based and larger windows, datasets with complete endpoint identifiers and timestamps for IoT traffic, the detection of unseen attack types using the uncertainty estimates, and streaming deployment.

Author contributions All authors contributed to the study's conception and design. Material preparation, data collection and analysis were performed by Srinivasa Rao Nune, Devi Uma Moka, G Gifta Jerith, Pattlola Srinivas and Sreelakshmi Udata. The first draft of the manuscript was written by Srinivasa Rao Nune, and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding No financial support was received by the authors in this research.

Data availability Data is available with the corresponding author and will be given on request.

Materials availability All materials are listed in the Data availability and Code availability statements.

Code availability The complete code (graph construction, models, baselines, evaluation and environment report) is deposited at Zenodo: https://doi.org/10.5281/zenodo.23293134.

Declarations

Competing interests The authors declare no competing interests.

Ethical approval This research does not involve humans or animals, so no ethical approval is required.

Consent for publication Not applicable.

Clinical trial number Not applicable.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Walling, S. & Lodh, S. An extensive review of machine learning and deep learning techniques on network intrusion detection for IoT. *Trans. Emerg. Telecommun. Technol.* **36**(2), e70064 (2025).
2. Al-Haija, Q. A. & Droos, A. A comprehensive survey on deep learning-based intrusion detection systems in Internet of Things (IoT). *Expert Syst.* **42**(2), e13726 (2025).
3. Noor, K., Imoize, A. L., Li, C. T. & Weng, C. Y. A review of machine learning and transfer learning strategies for intrusion detection systems in 5G and beyond. *Mathematics* **13**(7), 1088 (2025).
4. Fares, H. et al. Machine learning, deep learning and ensemble learning based approaches for intrusion detection enhancement. *EDPACS* **70**(1), 31–51 (2025).
5. Dong, H. & Kotenko, I. Cybersecurity in the AI era: analyzing the impact of machine learning on intrusion detection. *Knowl. Inf. Syst.* **67**, 3915 (2025).
6. Hozouri, A., Mirzaei, A. & Effatparvar, M. A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges. *Discov. Artif. Intell.* **5**(1), 314 (2025).
7. Deng, P. & Huang, Y. Edge-featured multi-hop attention graph neural network for intrusion detection system. *Comput. Secur.* **148**, 104132 (2025).
8. Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M. & Portmann, M. E-GraphSAGE: A graph neural network based intrusion detection system for IoT. In *NOMS 2022—IEEE/IFIP Network Operations and Management Symposium* (pp. 1–9). IEEE (2022).
9. Sarhan, M., Layeghy, S. & Portmann, M. Towards a standard feature set for network intrusion detection system datasets. *Mob. Netw. Appl.* **27**(1), 357–370 (2022).
10. Luay, M., Layeghy, S., Hosseininoorbin, S., Sarhan, M., Moustafa, N. & Portmann, M. Temporal analysis of NetFlow datasets for network intrusion detection systems (2025). arXiv preprint arXiv:2503.04404.
11. Mohammed, A. B. & Fourati, L. C. Investigation on datasets toward intelligent intrusion detection systems for Intra and inter-UAVs communication systems. *Comput. Secur.* **150**, 104215 (2025).
12. MadhloomKurdi, W. H., Alzuabidi, I. A., Najim, A. H., Kadhim, M. N. & Ahmed, A. A. Efficient two-stage intrusion detection system based on hybrid feature selection techniques and machine learning classifiers. *Int. J. Intell. Eng. Syst.* **18**(3), 224 (2025).
13. Ahmed, U. et al. Signature-based intrusion detection using machine learning and deep learning approaches empowered with fuzzy clustering. *Sci. Rep.* **15**(1), 1726 (2025).
14. Tahir, M., Abdullah, A., Udzir, N. I. & Kasmiran, K. A. A novel approach for handling missing data to enhance network intrusion detection system. *Cyber Secur. Appl.* **3**, 100063 (2025).
15. Kamalakkannan, D. et al. A detection of intrusions based on deep learning. *Cybernet. Syst.* **56**(5), 511–525 (2025).
16. Berguiga, A., Harchay, A. & Massaoudi, A. HIDS-IoMT: A deep learning-based intelligent intrusion detection system for the internet of medical things. *IEEE Access* (2025).
17. Berguiga, A., Harchay, A. & Massaoudi, A. HIDS-RPL: A hybrid deep learning-based intrusion detection system for RPL in internet of medical thing networks. *IEEE Access* **13**, 38404 (2025).
18. Susilo, B., Muis, A. & Sari, R. F. Intelligent intrusion detection system against various attacks based on a hybrid deep learning algorithm. *Sensors* **25**(2), 580 (2025).
19. Bamber, S. S., Katkuri, A. V. R., Sharma, S. & Angurala, M. A hybrid CNN-LSTM approach for intelligent cyber intrusion detection system. *Comput. Secur.* **148**, 104146 (2025).
20. Dash, N. et al. An optimized LSTM-based deep learning model for anomaly network intrusion detection. *Sci. Rep.* **15**(1), 1554 (2025).
21. Xue, Y., Kang, C. & Yu, H. HAE-HRL: A network intrusion detection system utilizing a novel autoencoder and a hybrid enhanced LSTM-CNN-based residual network. *Comput. Secur.* **151**, 104328 (2025).
22. Selem, M., Jemili, F. & Korbaa, O. Deep learning for intrusion detection in IoT networks. *Peer-to-Peer Netw. Appl.* **18**(2), 22 (2025).
23. Bandarupalli, G. Efficient deep neural network for intrusion detection using CIC-IDS-2017 dataset. *In 2025 First International Conference on Advances in Computer Science, Electrical, Electronics, and Communication Technologies (CE2CT)* (pp. 476–480). IEEE (2025).
24. Chen, H. et al. Intrusion detection using synaptic intelligent convolutional neural networks for dynamic Internet of Things environments. *Alex. Eng. J.* **111**, 78–91 (2025).
25. Serrano, W. CyberAIBot: Artificial intelligence in an intrusion detection system for cybersecurity in the IoT. *Future Gener. Comput. Syst.* **166**, 107543 (2025).
26. Sharma, J., Kumar, K., Jain, P., Alfilh, R. H. & Alkattan, H. Enhancing intrusion detection systems with adaptive neuro-fuzzy inference systems. *Mesopotamian J. CyberSecur.* **5**(1), 1–10 (2025).
27. Kamal, H. & Mashaly, M. Enhanced hybrid deep learning models-based anomaly detection method for two-stage binary and multi-class classification of attacks in intrusion detection systems. *Algorithms* **18**(2), 69 (2025).

28. Olanrewaju-George, B. & Pranggono, B. Federated learning-based intrusion detection system for the internet of things using unsupervised and supervised deep learning models. *Cyber Secur. Appl.* **3**, 100068 (2025).
29. Sudha, C. & Bolla, S. Intelligent intrusion forecasting framework for distributed environment using federated learning. *Clust. Comput.* **28**(6), 356 (2025).
30. Kumar, R. & Swarnkar, M. QuIDS: A quantum support vector machine-based intrusion detection system for IoT networks. *J. Netw. Comput. Appl.* **234**, 104072 (2025).
31. Benmalek, M. & Seddiki, A. Particle swarm optimization-enhanced machine learning and deep learning techniques for Internet of Things intrusion detection. *Data Sci. Manag.* https://doi.org/10.1016/j.dsm.2025.02.005 (2025).
32. Manivannan, R. & Senthilkumar, S. Intrusion detection system for network security using novel adaptive recurrent neural network-based fox optimizer concept. *Int. J. Comput. Intell. Syst.* **18**(1), 37 (2025).
33. Sheelavant, K., Supriya, B. Y., Assudani, P. J., Mahato, C. B. & Suman, S. K. Ensemble learning-based intrusion detection and classification for securing IoT networks: An optimized strategy for threat detection and prevention. *J. Intell. Syst. Internet Things* **17**(2) (2025).
34. Shi, L., Yang, Q., Gao, L. & Ge, H. An ensemble system for machine learning IoT intrusion detection based on enhanced artificial hummingbird algorithm. *J. Supercomput.* **81**(1), 110 (2025).
35. Alalwany, E. et al. Stacking ensemble deep learning for real-time intrusion detection in IoMT environments. *Sensors* **25**(3), 624 (2025).
36. Yang, L., Yang, J., Wei, C., Niu, G., Zhang, G., Wang, Y., Chai, L., Xia, W., Guo, H., Zhang, S., Liu, J., Yin, Y., Peng, J., Ma, J., Sun, L. & Li, Z. FuzzCoder: Byte-level fuzzing test via large language model (2024). arXiv preprint arXiv:2409.01944.
37. Yang, L. et al. Real-time intrusion detection in wireless network: A deep learning-based intelligent mechanism. *IEEE Access* **8**, 170128–170139. https://doi.org/10.1109/ACCESS.2020.3019973 (2020).
38. Das, S., Majumder, A., Namasudra, S. & Singh, A. Intrusion detection using CTGAN and lightweight neural network for Internet of Things. *Expert Syst.* **42**(2), e13793 (2025).
39. Rahman, M. A., Francia, G. A. & Shahriar, H. Leveraging GANs for synthetic data generation to improve intrusion detection systems. *J. Future Artif. Intell. Technol.* **1**(4), 429–439 (2025).
40. Mahmoud, M. M., Youssef, Y. O. & Abdel-Hamid, A. A. XI2s-IDS: An explainable intelligent 2-stage intrusion detection system. *Future Internet* **17**(1), 25 (2025).
41. Kharoubi, K., Cherbal, S., Mechta, D. & Gawanmeh, A. Network intrusion detection system using convolutional neural networks: Nids-dl-cnn for IoT security. *Clust. Comput.* **28**(4), 219 (2025).
42. Wali, S., Farrukh, Y. A. & Khan, I. Explainable AI and random forest based reliable intrusion detection system. *Comput. Secur.* 104542 (2025).
43. Cerasuolo, F., Bovenzi, G., Ciuonzo, D. & Pescapè, A. Adaptable, incremental, and explainable network intrusion detection systems for internet of things. *Eng. Appl. Artif. Intell.* **144**, 110143 (2025).
44. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P. & Bengio, Y. Graph attention networks. In *International Conference on Learning Representations (ICLR)* (2018).
45. Kipf, T. N. & Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)* (2017).
46. Hamilton, W. L., Ying, R. & Leskovec, J. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems* 30 (pp. 1024–1034) (2017).
47. SaiRamesh, L., Kathrine, G. J. W., Sathiyavathi, V., Selvakumar, K. & Sabena, S. Intelligent intrusion detection and prevention system for IoT using game theoretic approach. *Wirel. Pers. Commun.* **140**(1), 467–482 (2025).
48. Thaljaoui, A. Intelligent network intrusion detection system using optimized deep CNN-LSTM with UNSW-NB15. *Int. J. Inf. Technol.* 1–17 (2025).

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Srinivasa Rao Nune[1] · Devi Uma Moka[1] · G. Gifta Jerith[2] · Pattlola Srinivas[3] · Sreelakshmi Udata[4]

✉ Srinivasa Rao Nune
srinivas.nune@gmail.com

[1] Department of CSE-Data Science, Malla Reddy Engineering College for Women, Maisammaguda, Dhulapally, Post Via Kompally, Secunderabad, Telangana 500100, India

[2] AIML, Malla Reddy University, Maisammaguda, Dulapally, Hyderabad, Telangana 500100, India

[3] Department of CSE, Malla Reddy (MR) Deemed to be University, Maisammaguda, Medchal-Malkajgiri, Telangana 500100, India

[4] Department of OBGY, Malla Reddy Medical College for Women, Malla Reddy Vishwavidyapeeth, Suraram, Hyderabad, Telangana 500055, India